Serverless Avionics: A new Architectural Approach

Bojan Lukić* and Tim Schubert † and Sven Friedrich‡ and Cem Önem§ and Janick Beck¶ and Nora Breitmoser-Widdecke‖ and Alexander Ahlbrecht** and Wanja Zaeske†† and Umut Durak‡‡ and Harro von Viebahn§§
German Aerospace Center (DLR), Braunschweig, Germany

This paper presents Integrated Serverless Avionics (ISLA), a novel avionics architecture inspired by serverless computing principles, designed to address the growing technology demands of modern aircraft systems. ISLA introduces a modular, data-driven architecture composed of stateless, granular Tasks that are dynamically scheduled with adaptive resource management for flexible and autonomous system operation. Central to the platform are abstractions for safe, high performance scheduling and I/O operations, as well as a platform-independent WebAssembly (Wasm) runtime to ensure portability across diverse embedded hardware platforms. The design leverages Model-Based Engineering techniques, including the Systems Modeling Language (SysML) v2 modeling, standardized libraries, and automated transformation tools. This methodology facilitates comprehensive system modeling, configuration, and streamlined code generation, thus reducing development effort and certification costs. The process supports verification workflows that accommodate rapid system updates, fault tolerance, and safety certification efforts such as DO-178C compliance. Overall, ISLA aims to enable next-generation avionics systems to adapt swiftly to evolving operational and technological requirements, supporting future proofing of aircraft software architectures. Ongoing validation work is aimed at future realization in operational environments.

I. Introduction

Innovations in aviation, including both in manned and unmanned segments, necessitate new avionics paradigms not only for safety but also for fast integration and updating of the emerging software-defined functions. The challenge is the swift evolution in software needs and hardware tech, prompting the industry to seek novel approaches and platforms to expedite certification of new functions and updates.

In response, the aerospace industry is exploring paradigms that facilitate flexible, scalable, and maintainable system designs. Serverless computing principles, which have their roots in cloud computing, offer promising avenues in aviation. Applied to avionics, serverless software platforms can enable application developers to deploy applications across ground-based servers, avionics bays, and even actuators as well as sensors. Such a platform can significantly reduce the need for application developers to address specific architectural needs, like smart actuators and sensors, or resolve safety requirements like redundancy. Developed to meet specified system-level safety requirements such as reliability, availability, and timeliness, serverless computing for avionics will promote reuse, allow for quick updates, and provide independence from hardware.

This paper introduces Integrated Serverless Avionics (ISLA)–an avionics architecture that adapts serverless principles to the aerospace domain. ISLA conceptualizes avionics functions as stateless, granular Tasks that are data-driven and independently scheduled, enabling a flexible and autonomous platform capable of meeting stringent safety and timing requirements. Key components of the architecture include a unified Data-Fabric that abstracts communication interfaces, a platform-independent runtime environment for portability, and automated resource management mechanisms supporting dynamic system adaptation.

*Research Associate, Institute of Flight Systems.
†Project Manager, Institute of Flight Systems.
‡Research Associate, Institute of Flight Systems.
§Research Associate, Institute of Flight Systems.
¶Research Associate, Institute of Flight Systems.
‖Research Associate, Institute of Flight Systems.
**Team Leader, Institute of Flight Systems.
††Team Leader, Institute of Flight Systems.
‡‡Group Leader, Institute of Flight Systems, and AIAA Associate Fellow.
§§Advisor, Institute of Flight Systems.

© 2026 AIAA. Personal use of this material is permitted. Permission from AIAA must be obtained for all other uses, in any current or future media, including reprinting/republishing this material or reuse of any copyrighted component of this work in other works.
AIAA 2026-2264. AIAA SCITECH 2026 Forum. January 2026. DOI: 10.2514/6.2026-2264

By integrating Model-Based Engineering (MBE) techniques, such as high-level modeling with the Systems Modeling Language (SysML), and transformation tools, ISLA aims to streamline system design, facilitate certification, and support rapid deployment of updates. The anticipated benefits of the proposed framework include improved system reusability, reduced certification effort, enhanced fault tolerance, and increased operational flexibility for next-generation avionics.

Although comprehensive validation and certification validation lie beyond the scope of this work, the proposed framework lays a foundation for future research and development. Here, we introduce the ISLA architecture, the components of the development framework, and the underlying technological foundation of the platform.

The remaining paper is structured as follows: In section II, the technology that is serverless avionics is explained in more detail. Here, the fundamental bricks of the platform are introduced in the context of the overarching ISLA platform. Section III describes the development process for the platform. This is done in an MBE context with additional software integration and deployment steps. The related work in the context of serverless avionics is presented in section IV before concluding the paper in section V.

II. Serverless Avionics

As the aerospace industry increasingly leans towards leveraging advanced computing paradigms to meet strict safety, reliability, and performance standards, the concept of ISLA emerges as a promising architectural framework. ISLA aims to redefine how avionics software is designed, integrated, and managed for modern aircraft systems with its principles rooted in those of serverless computing.

Serverless

Serverless computing is an execution model where the cloud service provider dynamically manages the allocation of compute resources of the server, leading to a pay-as-you-go model where the consumer is charged only for the actual resources consumed [1]. Usually, it is implemented as Function as a Service (FaaS), where applications are composed of granular, light weighted, and stateless functions invoked by events (triggers) [1, 2]. The provider handles the management of the data center, server, and runtime environment, making the execution environment transparent to the developer [1]. Due to functions being stateless, maintaining application state across multiple invocations requires external mechanisms, typically backend services such as databases [3]. These granular functions are executed within virtualized sandboxes, such as containers or Virtual Machines (VMs), which form the execution unit for FaaS applications, ensuring isolation [2].

Because this paper discusses a specialized type of serverless architecture, that is, a serverless avionics architecture, the execution model of serverless computing is not discussed in further detail here. We encourage the reader to refer to the aforementioned sources.

Traditional avionics architectures, such as the widely adopted Integrated Modular Avionics (IMA) [4], rely on static, partitioned systems that are designed to guarantee temporal and spatial isolation among applications. These partitions are typically implemented using dedicated hardware or software domains that are statically configured and require rigorous certification processes. However, the increasing complexity of traditional avionics systems introduces significant challenges. These systems are evolving to support more advanced functionalities, such as Multi-Core Processor (MCP) or specialized components, which leads to issues like interference channels and unpredictable behaviors. These factors undermine the predictability and certifiability of traditional avionics designs, making it more difficult to ensure reliable performance in critical applications.

In response to these challenges, ISLA introduces a serverless computing inspired paradigm. This approach replaces the monolithic model of stateful partitions with a compositional one. In this model, avionics functions are encapsulated as granular, stateless Tasks which follow a stateless restart approach, replacing the stateful preemptive scheduling of traditional partitions. A Task receives all input data prior to its execution; only after it terminates its outputs become available to other Tasks. The execution of a Task is driven by data availability yet strictly bounded by its allocated time slice and deadline. These properties simplify the analysis and verification of system behavior, providing a different approach for achieving certifiable temporal behavior and managing interference channels [5].

The key to realizing ISLA lies in designing a system where these Tasks interact within a data-driven, explicitly modeled data flow, without extensive reliance on runtime operating system calls. This approach improves predictability by minimizing interference through mechanisms such as Logical Execution Time (LET), resource-pattern-aware scheduling and opportunistic slack time utilization. Such a system supports parallel and concurrent execution, aligning well with the capabilities of MCP hardware while simplifying system verification.

At its core, the ISLA platform consists of four components which run on every ISLA processing Node. In ISLA, Nodes bear the same meaning as modules in an IMA context. In the following, the four components are shortly introduced:

Orchestrator–The central control and configuration entity within the system. It maintains a static configuration that specifies which Tasks are to run, along with associated parameters like timing, data dependencies, and resource budgets.
Task-Dispatcher–The runtime execution manager for individual Tasks. It prepares the execution environment, which includes setting up memory regions, managing memory mappings, and enforcing time budgets.
Data-Fabric–The central communication abstraction for Tasks. It stores and transmits all input and output data of Tasks. It also facilitates the communication between Tasks of different Nodes by interfacing with the core network. It creates a safe coupling of Input/Output (I/O) operations to the static schedule, fulfilling each Task's data dependencies.
WebAssembly (Wasm) Interpreter–To support portability, ISLA deploys the platform-independent Wasm bytecode format. A Wasm Interpreter provides a sandboxed execution environment that can run platform-independent Task executables.

Figure 1 gives a high-level overview of the four components in an ISLA system. The runtime environment consists of the Orchestrator along with its Task-Dispatcher and the Data-Fabric.

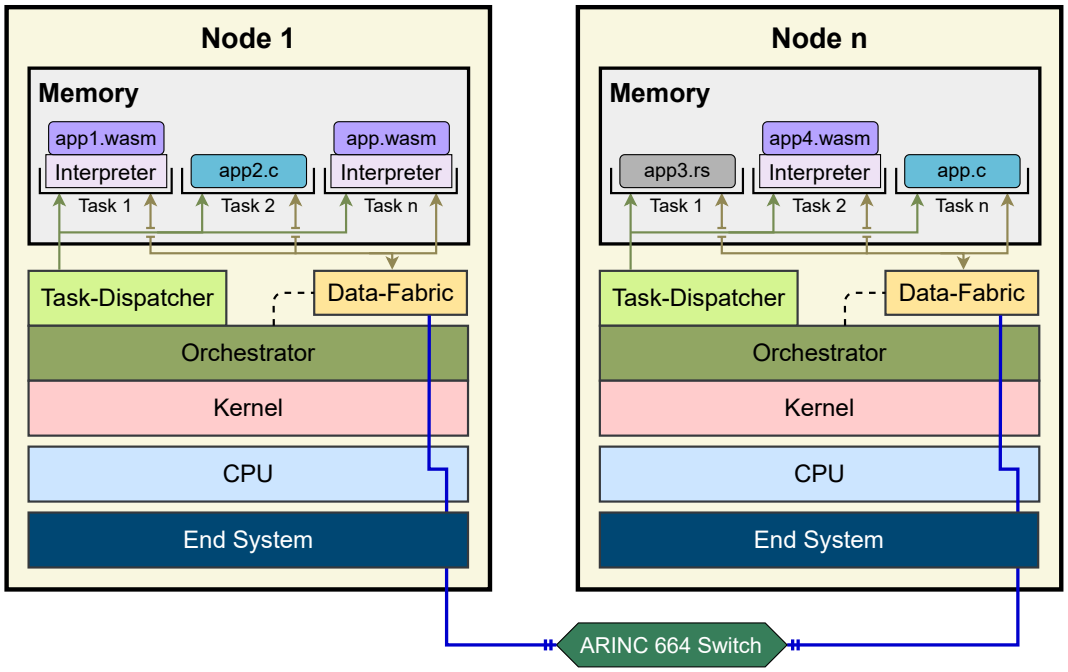

Fig. 1 Exemplary display of a composition of Nodes with interfaces for the ISLA platform.

The Task-Dispatcher and the Data-Fabric are libraries which can be used by the Orchestrator. All runtime components are deployed once per Node with different Tasks running on one or more Nodes. Data is exchanged between Nodes through a network backbone, in this case ARINC 664 [6]. The Orchestrator, Task-Dispatcher, Data-Fabric, and Wasm Interpreter are described in more detail in the following subsections. Their fundamental operation is described in high detail in the corresponding publications [5, 7, 8].

A. Orchestrator

At the heart of the ISLA architecture lies the Orchestrator, responsible for the high-level configuration, coordination, and management of system Tasks within the avionics platform. It draws inspiration from orchestration layers prevalent in cloud computing paradigms which handle complex interactions and execution graphs in serverless workflows.

The Orchestrator uses a configuration, defined at design time, which describes execution parameters along with Task execution planning. During runtime, the Orchestrator acts as the central control mechanism and communicates to the

Task-Dispatcher when a Task is due to be executed. It therefore manages execution according to some predetermined schedule and governs the system's runtime behavior. Moreover, the Orchestrator fulfills following functionalities:

Configuration Management–It maintains a comprehensive, static configuration that describes the entire system's structure. This configuration details which Tasks are to run, their I/O data dependencies, timing constraints, resource allocations, and execution parameters. This configuration is typically constructed during system development and verified as part of the certification process.
Task Scheduling Discipline–Based on the static configuration, the Orchestrator provides different scheduling disciplines. For most critical systems, static scheduling is utilized where Tasks are run in fixed predetermined time windows only. For systems more prone to interference like multi-core systems, opportunistic slack utilization is used. This mechanism utilizes accumulated slack time, i.e., unused execution capacity of preceding Tasks, as a safety net to absorb execution time variation. This allows subsequent Tasks to start early while preserving their original deadlines. As a final discipline, the Orchestrator schedules Tasks fully dynamic based on their I/O data dependencies and timing constraints. This discipline is only applicable for low DAL (Design Assurance Level) applications.
System Initialization and Decommissioning–During system startup, the Orchestrator loads the configuration, initializes the necessary parameters, and sets up the environment. Conversely, it manages graceful shutdowns, resource releases, and system reconfiguration if needed.

The design of the Orchestrator adheres to several key principles aligned with the goals of predictability, certifiability, and modularity.

Static, Deterministic Configuration–The Orchestrator relies on a configuration that is established during system design and remains largely unchanged during runtime, aside from potential controlled reconfigurations.
Minimal Runtime Overheads–To ensure real-time performance and reliability, the Orchestrator minimizes dynamic decision-making during execution. Its primary function is to schedule Tasks based on a predetermined schedule, reducing the risk of jitter or non-deterministic behavior.
Event-Driven Activation–For the fully dynamic scheduling discipline, a Task's execution can be triggered through events like the availability of input data, timer, health events, devices and more.
Multi-Core compatibility–When executing on MCPs one Orchestrator will be used per core, narrowing the scope of responsibility for minimal interference [5].

B. Task-Dispatcher

The Task-Dispatcher is a component that operates under the guidance of the Orchestrator, translating high-level configuration plans into execution environments for individual Tasks. Its primary role is to prepare and manage the runtime context necessary for Tasks to execute predictably, efficiently, and within strict safety and timing constraints. This component acts as the bridge between the static, schedule-driven orchestration logic and the execution-ready environment for Tasks. The Task-Dispatcher embodies a set of responsibilities that ensure Tasks are executed correctly:

Memory and Environment Preparation–Before a Task executes, the Dispatcher prepares its execution environment. It sets up the necessary memory mappings for inputs and outputs and ensures data is correctly loaded or transferred into the execution environment.
Task Activation and Management–The Dispatcher initiates Task execution, ensuring appropriate setup procedures are completed beforehand. After execution, it handles cleanup, data collection, and resource deallocation or reuse.
Resource Allocation and Timing Enforcement–The Dispatcher enforces strict resource boundaries, such as memory limits and time budgets, ensuring each Task operates within predefined safety margins.
Data Buffering and Data Flow Control–It manages input and output buffers, ensuring data dependencies are respected, and data is correctly transferred between Tasks and external modules or Data-Fabrics.

The implementation of the Task-Dispatcher adheres to principles that emphasize modularity and compliance with avionics safety standards:

Pre-allocated, fixed resources–Tasks do not perform dynamic memory allocation on system-wide memory. Instead, the Dispatcher allocates fixed memory regions during setup, reducing nondeterminism caused by heap management, fragmentation, or unpredictable allocation delays.
Support for Hardware and Platform Independence–The Dispatcher is designed to operate with platform-independent bytecodes or native code, maintaining flexibility in execution while supporting various Task implementations.
Synchronization and Scheduling Boundaries–By coordinating with the Orchestrator, the Dispatcher ensures Tasks are activated as scheduled, respecting timing constraints and deadlines, thus preserving system temporal integrity [5].

C. Wasm Interpreter

Traditionally, avionics software is compiled for specific hardware architectures, leading to costly revalidation and recertification whenever hardware changes occur. This effort can be circumvented through the use of a VM, provided that the bytecode it interprets, i.e., Executable Object Code (EOC), is now the centerpiece of certain certification objectives. Through this approach, hardware-specific compilation and certification effort is delegated to the implementation/porting of the VM itself. The application developer, on the other hand, simply compiles the application to this bytecode and runs it on a VM on any arbitrary hardware: as the certification evidence for the unchanged byte-code becomes more amenable to reuse, the recertification efforts would be substantially diminished in case the architecture changes.

Another advantage of employing a virtual instruction set architecture is the broadened prospects of non-intrusive code instrumentation, that is, instrumentation of EOC without making any change to program code or observable state. Depending on the interface offered by the Interpreter of the VM, classic implementations of per-instruction hooks, such as debuggers and coverage testers, already align with many verification schemes in DO-178C [9]. Another notable use case is fuel metering, where the Interpreter decrements a fuel counter every time it executes an instruction and stops the application if it depletes the fuel that was supplied on startup. In the context of avionics, the fuel concept can be employed to fortify worst-case execution time claims for various VM applications [8].

Wasm is an excellent candidate as a bytecode format for avionics where safety-critical systems demand rigorous certification and high reliability [7]. The corresponding VM is abetted by a formal specification which is beneficial when demonstrating compliance with standards like DO-178C. The typed stack and the strict control flow the VM stipulates, along with its Harvard memory model with no forced use of a garbage collector, are all favorable when certification is involved. This respectively concerns objectives regarding type punning, structured control flow, unwanted (function) pointer manipulation and heap memory management.

Embedding a Wasm Interpreter into a ubiquitous Task variant in the ISLA solution stack would abstract away a significant portion of the problem of integrating Nodes with distinguished architectures, as they simply run the same bytecode. The most pragmatic implementation strategy for a Wasm application to interface with the underlying platform is with an Application Programming Interface (API). This would comprise a set of synchronous Wasm functions that the application links to through import directives. The import items could be specific utilities, e.g., file operations or networking, that necessitate the involvement of the underlying platform. However, this approach would couple the underlying platform to the API, thus the application itself, rendering the serverless concept moot. An approach that uncouples the Wasm Interpreter operations from the runtime involves mandated signatures for all Wasm applications that run on ISLA and an intentional omission of an API to link to. That is, the Wasm applications that run on ISLA have no platform specific functions to call. Instead, the applications consume their inputs, which are statically declared in the Task configuration items, assuming they are readily available through the Wasm runtime objects they own. This may be input arguments in their entry point function or their Wasm linear memory. This availability can be satisfied in turn by the underlying platform through a modification of the Interpreter instance, before the application is invoked.

Although the use of a Wasm Interpreter provides portability and alleviates certain recertification burdens, it also implicates a considerable performance penalty. This performance penalty is caused by execution of virtual instructions in software. A second concern is the availability of an avionics oriented toolchain: being a language agnostic bytecode format, Wasm already features C, C++, and Rust front ends in the LLVM ecosystem. However, no qualified compiler of Wasm for safety-critical use, such as CompCert [10], exists yet.

D. Data-Fabric

The Data-Fabric performs implicit communication between Tasks of the same Node but also between Tasks of different Nodes.

Since Tasks follow the restart model, they have no persistent state with no explicit communication mechanism existing between them. Any input and output data needs to persist somewhere outside a Task. On each ISLA node, a single centralized data store takes on this role. Data inside the store is addressed using labels that identify the channel that holds the data. The store implements two kinds of data channels which offer similar semantics to ARINC 653 [11] channels:

- **Sampling channels** store the most recent sample of some data. Only the most recent sample can be accessed and reading from the channel does not consume its contents.
- **Queuing channels** implement a single-producer-single-consumer First In, First Out (FIFO) processing method with either drop-first or drop-last semantics.

To ensure spatial and temporal isolation, Tasks have no way of accessing the store themselves. Instead, they passively participate in a safe shared memory protocol that is managed by the Data-Fabric. Before the Task is executed, the Data-Fabric reads a Task's required inputs from the store's channels into the private memory of the Task. Then it sets up memory regions in which the Task can later store its outputs. After the Task finishes successfully, the Data-Fabric collects these outputs and places them into the appropriate channels inside the store.

Which channels the Data-Fabric uses for obtaining inputs or storing a Task's outputs is strictly defined as part of the platform configuration, along with the order in which Tasks are executed. This way, the indirect data-flow between Tasks is tightly coupled to the execution schedule and lends itself easily to formal reasoning and verification.

As an example, the Data-Fabric could explicitly define all interactions of a set of Tasks with a multiple-producer multiple-consumer queue. Since the platform configuration explicitly defines the producing and consuming Tasks and the order in which they are executed, it also defines the sequence in which queue items are consumed and produced. The same way, the platform configuration can be statically verified to ensure that all queues adhere to single-producer-single-consumer semantics.

Additionally, since all of a Task's input data is available to it before it executes and its output data is read only after it has finished, LET analysis [12] is simplified. The basic concept of the Data-Fabric is illustrated in Figure 2.

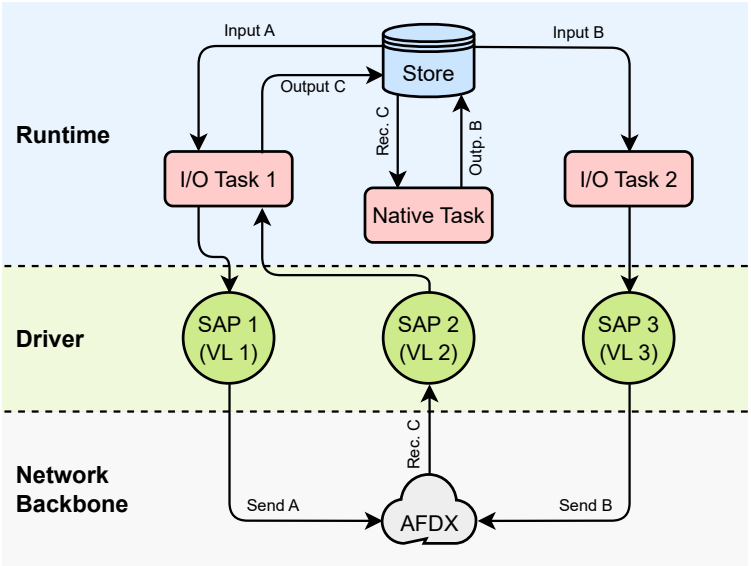

Fig. 2 High-level overview of information flow through the Data-Fabric.
SAP–Service Access Point; VL–Virtual Link; AFDX–Avionics Full-Duplex Switched Ethernet.

Networking Support

Between individual Nodes, ISLA uses industry standard Ethernet networking such as ARINC 664 [6] or IEEE Time-Sensitive Networking (TSN) [13]. Communication with networking hardware is handled in dedicated I/O-Tasks which form a safe barrier between a potentially asynchronous network and a potentially hard real-time Task schedule. Same as regular Tasks, I/O Tasks are scheduled as part of the static platform schedule. They exchange input and output data with a predefined set of input and output channels of the store. However, I/O Tasks' inputs are sent to the networking hardware and outputs are received from it. This way, the schedule can be used for expressing when which data is due to be sent to the network and when it is due to be received from the network. In contrast to most application-specific Tasks, I/O Tasks only copy data between the hardware and the store and can be reused in a variety of configurations.

Use of Interrupt Coalescing

In real-time systems, I/O operations are often required to have bounded latency, since the system should be responsive enough to handle certain critical events. To ensure the timely processing of I/O events, peripheral devices use hardware interrupts to preempt the execution of the host system. In certain conditions, this may alter the execution of the static schedule or lead to deadline misses which, in general, is not acceptable for a hard real-time system.

Interrupt coalescing can be used to alleviate this effect, while allowing for a reasonable trade-off between performance and latency. It is often implemented by first storing interrupts of concern in a hardware FIFO and sending actual hardware interrupts to the host system according to some predefined policy, such as a periodic timer. In the case of ISLA, this policy could be made to align interrupts from the networking hardware with the host system and its static schedule, so that these interrupts of concern need only be processed during the statically assigned I/O-slots. Interrupts of the host system, such as for interrupting Tasks that overran their deadline, would be excluded from this policy, since the coalescing happens in the networking hardware.

Relation to the System Model

The Data-Fabric enforces a coupling between data dependencies and the static schedule. This coupling is introduced by the system model, which defines at what point which data should be made available to the network and to Tasks of the same Node. It also defines at what point in time each Task requires each data.

This set of constraints is first introduced in the high-level system model, expressed through the low-level target configuration, and implemented by the Data-Fabric and Orchestrator.

III. Serverless Avionics Development Process

The development process for the ISLA system strongly aligns with the platform's overarching principles of modularity, short update paths, and overall effort reduction. In this section, the different development stages are presented in separate subsections, with an emphasis on MBE of the overall platform. This concerns different phases, from high-level system modeling and configuration as well as code generation to the point of the deployment of generated artifacts on the target platform. These phases are shown on a high level of abstraction in Figure 3.

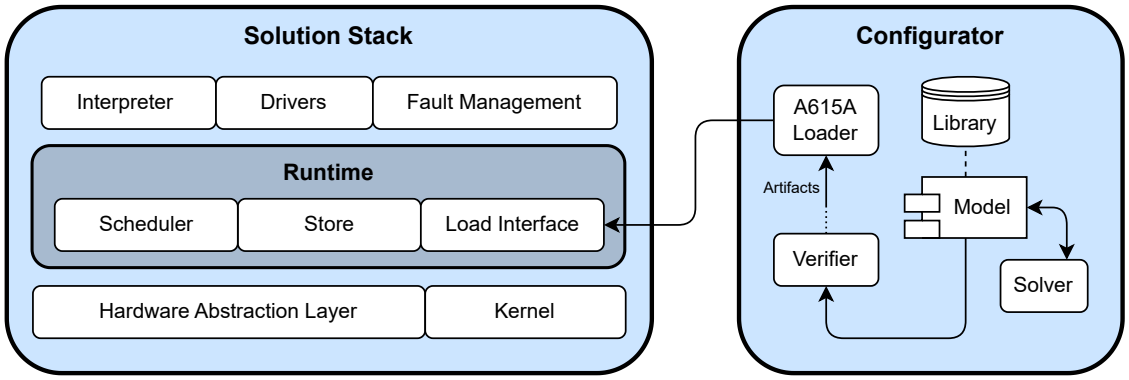

Fig. 3 High-level overview of the processes involved for developing and setting up the ISLA platform. The previously defined technologies are part of the solution stack.

The primary development process applies the MBE method using a Single Source of Truth (SSOT) architecture. The modeling environments consist of multiple tools, including those for Model-Based Systems Engineering (MBSE), Model-Based Software Engineering (MBSwE), and Model-Based Verification (MBV). The MBV tools implement the Functional Mock-up Interface (FMI) standard [14] to support model exchange and uniform simulation definitions.

All modeling activities–system architecture modeling and functional modeling–are constrained by predefined structures stored in ISLA libraries. Each modeling environment uses its dedicated library. A dedicated solver operates on the architecture model. It optimizes the allocation of predefined Tasks to available Nodes based on platform resources and generates a valid schedule for these Tasks. Subsequent verification activities check compliance with the requirements defined in the architecture model. Information is propagated only from a valid model instance.

The modeling environments support model transformations via standardized interfaces or dedicated transformation tools. For example, functional models can be transformed into application source code, which may be manually adapted or used without modification. Additional transformations convert the system architecture model into a low-level configuration format. Together with all EOC, these artifacts are loaded onto the target platform during deployment.

Figure 4 provides a detailed overview of the development process within the broader MBE context and its separate development workflows.

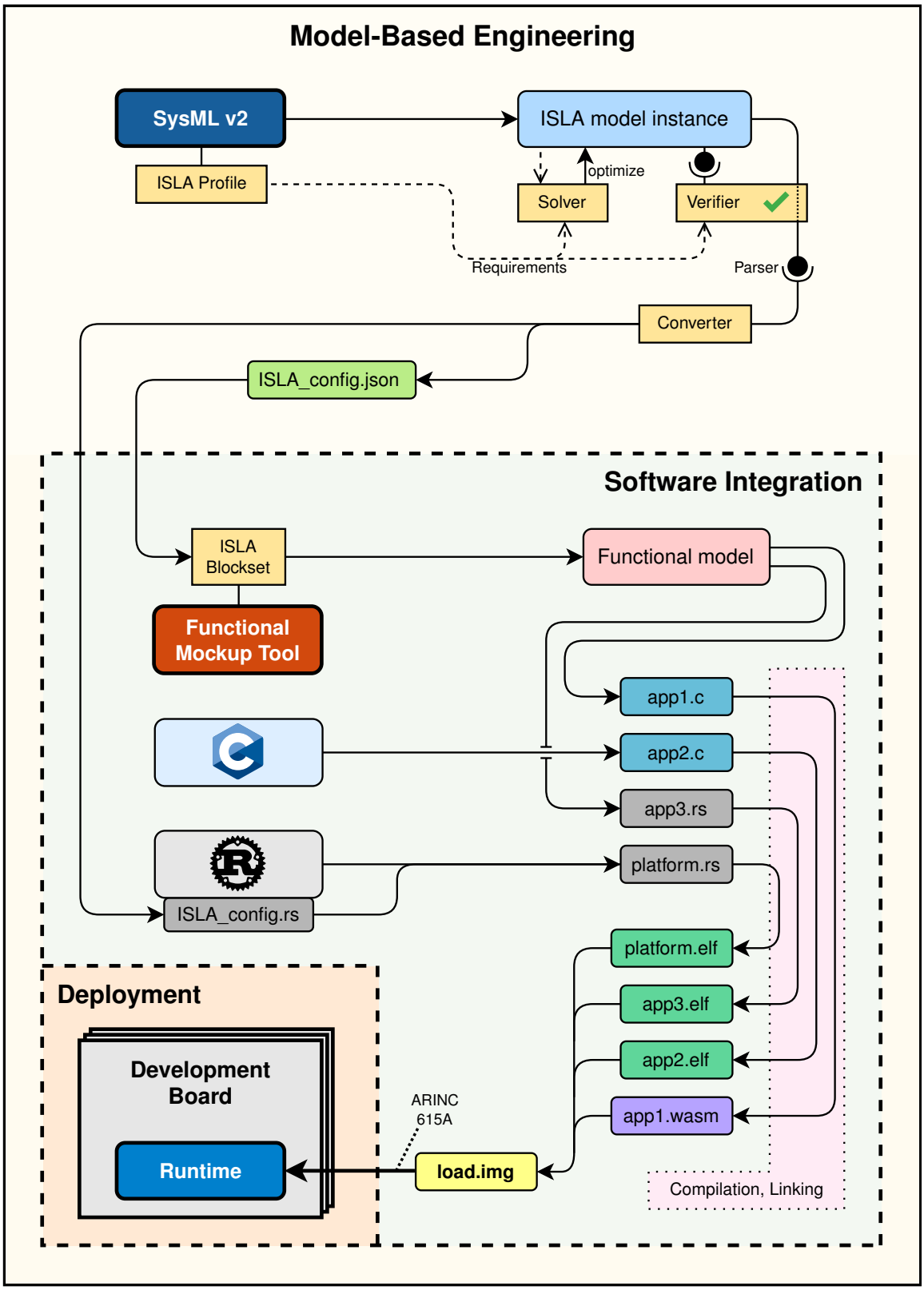

Fig. 4 Information flow for processes, tools, and generated artifacts in the ISLA development toolchain displayed from a holistic MBE perspective.

The single development steps and artifacts in Figure 4, depicted as atomic elements, are selectable and link to the respective content and appendix in this paper. We invite the reader to select any elements of interest to jump to the desired content in the paper. The detailed description of the development phases are discussed in detail in the following sections.

A. SysML v2 Model

The use of the SysML v2 modeling language for systems engineering is intentional. SysML v2 introduces dynamic modeling capabilities, including state machines, constraint-based system analysis, and system verification. A key advantage of SysML v2 is its textual model representation. The benefits of textual notation for system modeling are discussed later in this paper.

To improve model standardization, a dedicated library for the ISLA platform is defined. This library provides stereotypes for system components that function as blueprints and support reuse across multiple model instances. The blueprints also ensure that system designs follow predefined structural constraints. An excerpt of the ISLA library, expressed in SysML v2, is shown in Listing 4. A corresponding model instance in textual notation, using the ISLA library definitions, is provided in Listing 5. The approach of defining profiles, i.e., libraries for standardized system development, aligns with the methodologies described in [15–17].

A graphical representation of the model instance equivalent to the textual notation in Listing 5 is shown in Figure 5.

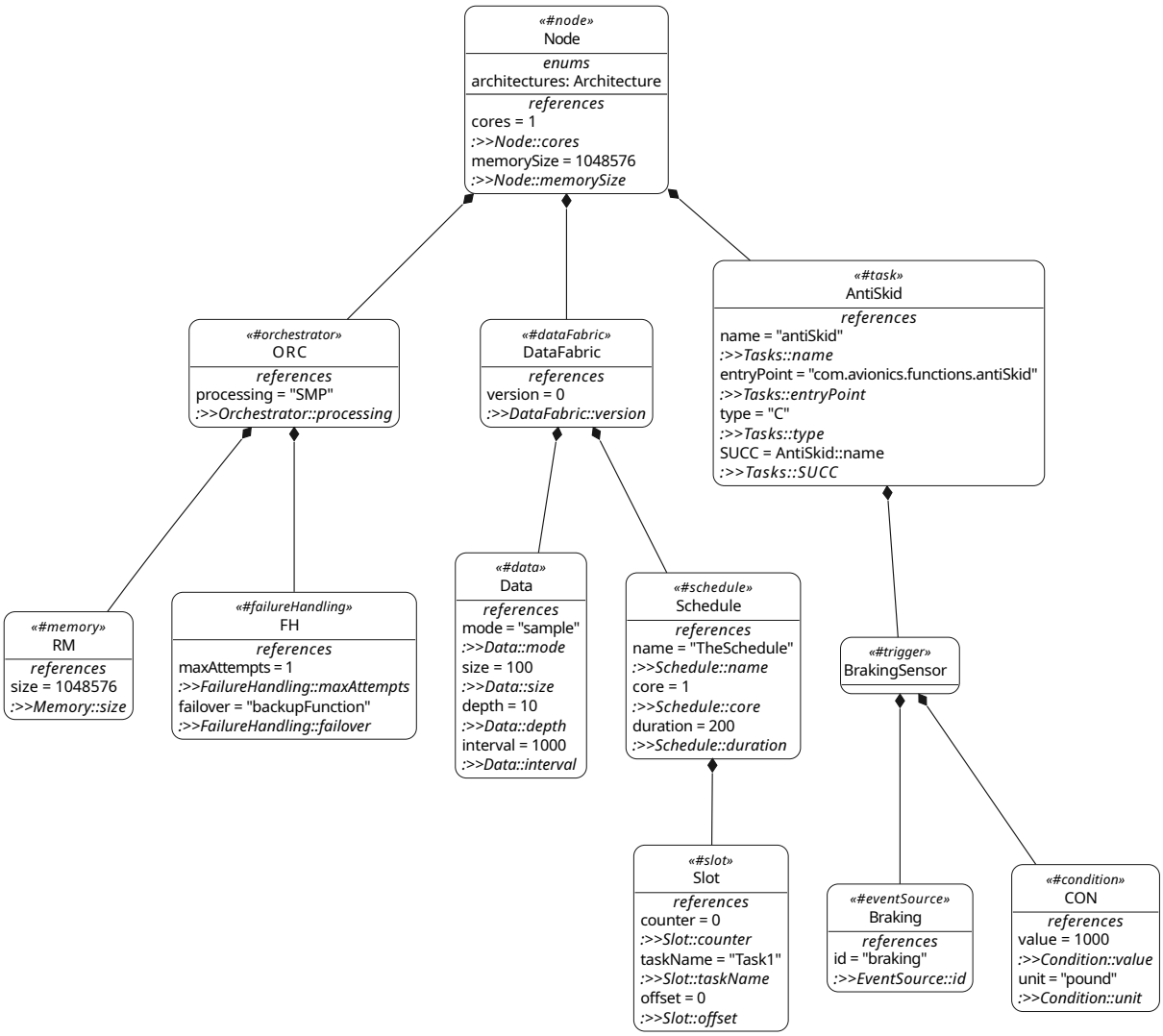

Fig. 5 Visual representation of a sample ISLA platform instance modeled with SysML v2.

B. Solver

In ISLA, architecture modeling may not inherently be function-driven. This contrasts with conventional avionics development approaches such as IMA, where all required functions and corresponding system requirements are defined at design time, and the system architecture is subsequently derived from them [18–20]. In ISLA, Tasks are also specified at design time. However, they are specified as collections of functions with constrained properties, including execution times and inter-Task dependencies. The complete set of Tasks in a system instance can therefore be represented as a directed graph, which may contain cycles. Given a predefined set of Nodes, Tasks must be allocated to Nodes in a manner that satisfies their timing constraints and dependency relations. Because this allocation is not performed at design time, the architecture model requires post-modeling optimization. For this purpose, a solver is applied: it ingests the model, computes an optimized allocation, and updates the model upon successful completion.

The core of the problem can be seen as a multi-objective optimization problem that combines both Task allocation and Task scheduling in a distributed system:

1) **Task Allocation**: Distribute Tasks across Nodes in an optimal way, respecting constraints such as dependencies, communication latency, and time for execution.
2) **Task Scheduling**: Find an optimal schedule for these Tasks, considering their periodicity requirements, communication latency, and system architecture constraints.
3) **Path Assignment**: Find valid paths through the network backbone, considering inter-Task dependencies and communication latency.

An existing model that closely relates to the problem statement in ISLA is the Flexible Cyclic Jobshop Scheduling Problem (FCJSP) [21]. The FCJSP involves the periodic scheduling of jobs on machines capable of performing multiple types of operations. The objective is to determine job assignments and processing sequences that minimize the cycle time, defined as the duration between successive completions of identical jobs. This problem extends traditional jobshop scheduling by incorporating machine flexibility, allowing each machine to handle various job types, thereby increasing scheduling complexity. The FCJSP is characterized by precedence constraints and the need to optimize both Task-to-machine assignment and temporal sequencing within a cyclic framework. Given its combinatorial nature, solving the FCJSP requires advanced optimization techniques.

A minimalist description of the problem for ISLA may be as follows: Given a set of Tasks $T = t_1, \ldots, t_n$ with end-to-end latency, dependencies, and periodicity requirements, allocate and schedule these Tasks across a set of Nodes $N = n_1, \ldots, n_k$ such that the total execution time is minimized, all precedence constraints $t_i \rightarrow t_j$ are respected and the cyclic schedule is feasible.

This problem can be solved using combinatorial optimization methods such as Mixed Integer Linear Programming as well as Integer Programming, or metaheuristics like Genetic Algorithms and Simulated Annealing.

C. Verifier

Verification is a systematic process used to ensure that a system or its elements meet specified requirements, perform intended functions, and conform to design characteristics. It involves a series of planned actions, such as inspection, testing, demonstration, and analysis, which are performed in parallel with system definition and realization processes. The goal of verification is to mitigate risks and avoid costly rework. Verification activities are typically defined early, during requirements analysis, and are guided by a verification matrix that specifies how and when each requirement will be verified. These activities may involve various methods, such as:

- **Demonstration** for qualitative observations
- **Examination** of design data and physical components
- **Analysis** using recognized techniques or simulations
- **Testing** under controlled conditions to gather quantitative data

Verification is performed at all stages, from individual system elements through to the complete integrated system, ensuring that the system meets its functional and performance requirements [22–24].

In the context of system certification and qualification, various strategies can be employed to ensure that a system meets its design specifications. A fundamental responsibility of the system developer is to guarantee that the final system adheres to the intended design and performance criteria. This can be achieved by verifying both the correctness of the system artifacts loaded onto the platform and the tools used throughout the development process. For complex systems such as ISLA, this often entails the qualification of multiple development tools. However, a more efficient approach involves the deployment of a dedicated verifier at a specific stage in the development toolchain. This strategy

ensures that only the verifier itself requires qualification, rather than the entire suite of tools. In such a framework, the tools used to generate system representations do not need to be individually qualified, as long as the verifier, which is responsible for validating the system's representations, is qualified. Specifically, in the ISLA development process, the high-level SysML v2 SSOT model instance is subjected to verification before any subsequent dissemination or utilization of the model data. This approach significantly reduces the qualification burden by isolating the qualification responsibility to a single, well-defined element in the toolchain.

Some external tools, which can be used as verifiers, offer functionalities for performing verification directly on SysML v2 models. One exemplary tool, SysIDE, offers capabilities for modeling, simulation, verification, and validation of SysML models. The potential functionalities required for system verification can be summarized as follows:

- Consistency Checks: verify that all elements of the SysML model, e.g., parts or interfaces, are logically consistent and conform to defined relationships.
- Automated Model Validation: fulfill predefined rules or requirements, including functional and non-functional constraints such as performance and safety.
- Formal Verification: perform exhaustive model checking, identifying potential logical errors or inconsistencies in system behavior, especially in complex models involving, for instance, state machines.
- Model Comparison: detect changes across different versions of the SysML v2 model, helping ensure that updates do not introduce errors or invalidate previous verification results.
- Automated Testing: generate test cases based on behavior models, e.g., state machines, to assess system responses under various conditions, facilitating early detection of design flaws.

A sample requirement for the ISLA platform is shown in Listing 1. Tools, like the aforementioned SysIDE, perform verification against such requirements.

```
requirement taskFrequencyRequirement {
    subject task : Task;
    in frequencyActual :> ISQ::time;
    doc /* The task shall execute with a frequency
        * higher than or equal to 100 Hz, i.e.,
        * less than or equal to every 10 ms. */
    require constraint {
        frequencyActual == task.frequency and
        frequencyActual <= 0.01[SI::second]
    }
}
```

Listing 1 Sample requirement for the ISLA platform written in SysML v2. Verification can be performed against such requirements using tools such as SysIDE.

D. Model Transformation

Historically, the process of model transformation within the SysML ecosystem has been impeded by the complexities associated with model exchange formats. XML Metadata Interchange (XMI), although standardized for Unified Modeling Language (UML) models, often proved insufficient for seamless interoperability due to its lack of consistency across different tool implementations. Variations in textual representations, particularly when incorporating custom extensions and profiles, further complicated data exchange, rendering the process cumbersome and unreliable [25]. While there were some standardized parts to the first version of SysML, mostly from its share with the UML, the same models created in different modeling tools often differed enough to not be compatible with one another. This made model-to-text and text-to-text transformations–from textual model representations–cumbersome and non-generic for each domain and tool.

Within the ISLA platform development ecosystem, both the model-to-model and the model-to-text transformation play a central role. In both cases, a system model instance is used as the source for transformations. For functional development processes, the high-level SysML v2 model is transformed to a different model representation. At the same time, a high-level configuration format, used for other purposes, can be generated from the SysML v2 model instance. As mentioned before, SysML v2 as a source modeling language is used deliberately because it offers a standardized way to represent models textually. This standardized approach facilitates tool interoperability, reinforces a unified modeling

paradigm based on an SSOT architecture, and streamlines the generation of engineering artifacts. Moreover, the SysML v2 pilot-implementation comes with a native parsing framework, called XText. These two concepts combined provide a powerful way for performing the aforementioned model transformations. While the XText parser is well-defined for SysML v2 and tool-agnostic, it still requires a custom parsing logic, or library. For the ISLA platform, the parser follows the structure of the previously presented SysML v2 library. Once the parsing logic has been defined, the parser can be extended and scaled arbitrarily. An excerpt of the native SysML v2 parsing library developed for the ISLA platform is shown in Listing 6.

SysML v2's native XText-based parser inherently includes model verification capabilities. By parsing text according to the SysML v2 grammar and validating it against library definitions, it helps detect and report implementation mistakes. When combined with the SSOT methodology–where the textual representation serves as the foundational model for downstream engineering processes–this results in a highly effective modeling environment with preliminary validation functionalities. Working on the textual representation of a model for transformation directly has the benefit of quick serialization and information exchange. It is suitable for rapid prototyping and testing of systems. A different method is the transfer of a model snapshot to a database management system (DBMS), as presented in [26]. Outsourcing all model information, including meta data and unique IDs for atomic model elements, to DBMS and working on the data separately, comes with different benefits. With this approach, advantages in version control and model consistency arise. This is beneficial when working with numerous stakeholders and employing distributed development processes. In summary, both the direct parsing of data from a model as well as the method described in [26] are applicable for development of the ISLA platform and are beneficial for different use cases.

E. High-level Configuration Format

The high-level configuration in the ISLA development process is generated with a model-to-text transformation. This configuration is a model snapshot from the system architecture model. It contains all components defined and used in the architecture model as well as a valid schedule and the allocation of Tasks to existing Nodes. This configuration is an exchange format and can be used to set up other tools. In this example, the configuration is imported in an arbitrary Functional Mock-up Tool to automatically generate a functional system model. The functional model, in turn, can be used for simulation and code generation.

For the extraction of information from the SysML v2 model and the generation of a configuration, the previously defined XText parser is used. The high-level configuration is stored in JSON format and closely follows the structure from the architecture model. The benefits of JSON as a serialization format, specifically its cross-compatibility, are described in [27]. It contains general information about the system, such as global resources per Node or dependency information between different Tasks. The common high-level configuration follows the structure with parameter descriptions as listed in the following overview:

```
Node
├── architecture – CPU architecture on this Node.
├── cores – number of cores available on this Node.
├── memorySize – size of the memory available on this Node.
│
└── Orchestrator
    ├── processing* – in case of multi-core, type of processing.  That is, Asymmetric
    │   Multi-Processing (AMP) or Symmetric Multi-Processing (SMP).
    │
    ├── Memory
    │   ├── type – requirements for memory type, that is, RAM, flash, or I/O.
    │   └── size – requirements for memory size.
    │
    └── Failure Handling**
        ├── maxAttempts – maximum number of attempts for failure handling.
        ├── backoffStrategy – retry method in case of failure events.
        └── failover – backup function which is used in case of a failover.
```

↓ *continued on next page* ↓

*Only required if one Orchestrator oversees multiple cores on one Node.

**Dependent on dedicated health monitoring Task which needs to be instantiated separately for the system.

- **DataFabric**
  - **version** – version of the configuration.
  - **Data**
    - **mode** – type of data, that is, sampling or queuing.
    - **size** – size of data.
    - **depth** – depth of data.
    - **interval** – interval between each sampling of data.
  - **Schedule**
    - **name** – name of the schedule.
    - **core** – core on which the schedule will run.
    - **duration** – major duration of this schedule.
    - **Slots**
      - **counter** – ID of this slot.
      - **taskName** – Task scheduled in this slot.
      - **offset** – offset to the beginning of this schedule.
      - **duration** – duration of Task execution in this slot.
- **Tasks**
  - **name** – Task name.
  - **version** – Task version.
  - **executionMode** – mode of execution for the Task, that is, event-driven or scheduled.
  - **type** – Task type, that is, Wasm object code, compiled C/C++/Rust code, etc.
  - **PRED** – dependencies to other Tasks with their output being used in this Task.
  - **SUCC** – dependencies to other Tasks, which use this Task's output.
  - **input** – input that goes into this Task.
  - **output** – output provided from this Task.
  - **eteLatency** – maximum end-to-end latency from the start of the Task until its output; If 0, scheduling best effort.
  - **Triggers**
    - **Event Source**
      - **type** – type of the event trigger source, that is, sensor, remote data concentrator (RDC), legacy unit, etc.
      - **id** – unique ID of the event source.
    - **Condition**
      - **type** – condition type for event trigger.
      - **value** – value that, together with the condition type, dictates actions.
      - **unit** – unit of signal.

Additional configuration parameters, such as for the network backbone, can be considered. In this example they are omitted as the focus lays on the configuration of the solution stack for ISLA. An exemplary configuration for a network backbone such as AFDX is presented in [27].

F. Target Low-Level Configuration

As with the high-level configuration, a low-level configuration is generated from the architecture model using the provided parser and model converter. It is used to configure the target system. The low-level configuration is generated as source code for integration with the platform code. The low-level configuration is shown in Listing 2. The generated code contains a single constant-initialized static symbol that references the entire configuration for the target. This symbol is loaded by the root Task during system initialization. It uses it to initialize the components of the runtime.

The low-level configuration defines the system's architecture, Task allocation, data exchange, and scheduling in an explicit manner. It specifies the hardware resources, such as Nodes and cores, and assigns Tasks to particular processing elements, along with the associated executable images. Communication channels between Tasks are established via data

queues with specific size and depth. Here, depth refers to the number of data packets, messages, or I/O requests that are waiting in a queue to be processed or transmitted. The configuration also lays out the precise timing and execution order for each Task through schedules, including start times, durations, and specific time slots.

```
static CONFIG: &Configuration = &Configuration {
    version: Version("0"),
    nodes: &[(NodeName("Node"), Node { cores: &[Core(1)] })],
    tasks: &[(
        TaskName("AntiSkid"),
        Task::Native(Runnable {
            image: ImageName("antiSkid_elf"),
            inputs: &[DataName("data1")],
            outputs: &[DataName("data1"), DataName("data2")],
        }),
    )],
    data: &[
        (DataName("data1"), Data::Queue { size: 10, depth: 3 }),
        (DataName("data2"), Data::Queue { size: 10, depth: 3 }),
    ],
    schedules: &[(
        ScheduleName("schedule"),
        Schedule {
            node: NodeName("Node"),
            core: Core(1),
            duration: Duration(10_000_000),
            slots: &[Slot {
                task: TaskName("AntiSkid"),
                offset: Duration(0),
                duration: Duration(200_000),
            }],
        },
    )],
};
```

Listing 2 Exemplary low-level configuration generated from the high-level architecture model.

In this example, the low-level configuration is generated as Rust code. This configuration can then directly be integrated in the platform source code which is also written in Rust.

G. Functional Mock-up Interface

As previously mentioned, the high-level configuration format for the ISLA platform is used to disseminate data for subsequent development processes. In this context, a functional model is automatically generated from the information contained in the JSON configuration. Given the modular and open development methodology, the objective is to avoid reliance on a single proprietary functional design tool, instead enabling the flexibility to use any applicable tool for various functional modeling tasks. This is where the FMI standard plays a critical role by facilitating tool and model interoperability without vendor lock-in. The FMI standard defines a consistent interface between different tools, such as MATLAB/Simulink*, SCADE†, and Pictorus‡, ensuring cross-tool compatibility and vendor neutrality.

*MATLAB/Simulink, as presented in: `https://www.mathworks.com/products/simulink.html`

†Ansys SCADE Suite, as presented in: `https://www.ansys.com/products/embedded-software/ansys-scade-suite`

‡Pictorus, as presented in: `https://www.pictor.us/`

The FMI standard offers several benefits, including enhanced collaboration, co-simulation capabilities, and support for real-time simulation. It also promotes the integration and reuse of models across different domains. With FMI support, functional system models can be exported as Functional Mock-up Units (FMUs), which can then be imported into other simulation tools that adhere to the FMI standard [14]. For example, a Task developed in Simulink can be exported as an FMU and subsequently used in other simulation environments like SCADE or Pictorus. The transformation from the high-level configuration, i.e., the architecture model, to the functional model can be pragmatically achieved by parsing the information from the JSON configuration and mapping it to the corresponding components of the functional model. The set of parameters used for creating a minimal functional model for the ISLA platform may look as follows:

```
Node
├── architecture – CPU architecture on this Node.
├── cores – number of cores available on this Node.
├── memorySize – size of the memory available on this Node.
│
└── Tasks
    ├── name – Task name.
    ├── version – Task version.
    ├── executionMode – mode of execution for the Task, that is, event-driven or scheduled.
    ├── type – Task type, that is, Wasm object code, compiled C/C++/Rust code, etc.
    ├── PRED – dependencies to other Tasks with their output being used in this Task.
    ├── SUCC – dependencies to other Tasks which use this Task's output.
    ├── input – input that goes into this Task.
    ├── output – output provided from this Task.
    ├── eteLatency – maximum end-to-end latency from the start of the Task until its output; If 0,
    │   scheduling best effort.
    │
    └── Triggers
        ├── type – condition type for event trigger.
        ├── value – value that, together with the condition type, dictates actions.
        └── unit – unit of signal.
```

It is important to note that the information used for creating the functional system model often represents only a subset of the data contained in the full configuration, as redundant metadata–superfluous for functional modeling–may be present. Once the transformation is complete, the user can proceed with software integration tasks within the functional modeling environment. A library, analogous to the one defined for the SysML v2 ISLA model, is also established for functional modeling. This library, referred to as the ISLA Blockset, serves as a blueprint, guiding the modeler to design the functional system according to a predefined structure. The ISLA Blockset is conceptually similar to other libraries used for functional modeling, such as the ARINC 653 Simulink Blockset presented in [28].

The ISLA Blockset defines standardized libraries for system components. For example, a Simulink blockset may contain C header files that define standardized blocks, which can be reused across different models. The source code from these header files can also be reused for purposes such as code generation from the functional model. From the example of IMA, a minimal reference code used for a Task block could look as in Listing 3.

The code provided in Listing 3 is generated from the functional model and reflects the implementation of a specific Task. The FMI serves as the interface that makes this functional model compatible with a range of tools. Inside each FMU, the functional model of a Task is encapsulated. The code presented here is a direct output of this functional model. The FMU can be viewed as an implementation of the FMI interface, wherein at runtime, the code specified by the functional model is executed according to the interface's specifications[§].

Furthermore, the functional model libraries, such as the ISLA Blockset, are not limited to a single tool but can, under certain conditions, be exchanged between different functional modeling tools. This means that not only can functional models be transferred as FMUs across different simulation environments, but the underlying libraries used for creating these models can also be interoperable between tools.

[§]FMI standard header functions are presented in the following repository:
`https://github.com/modelica/fmi-standard/blob/main/headers/fmi3Functions.h`

```c
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint8_t *const addr;
    const size_t len;
    const bool writeable;
} Descriptor;

typedef struct {
    const size_t idx;
    size_t len;
} State;

typedef struct {
    Descriptor *descriptors;
    State *inputs;
    State *outputs;
} States;

void _start(States *states) {
    // Get the first input state's location and length
    State input_state = states->inputs[0];

    // Get input state data
    uint8_t *const input_data = states->descriptors[input_state.idx].addr;
    const size_t input_data_len = input_state.len;

    // Hook for code from model logic contained in Functional Mock-up Unit
    ...
}
```

Listing 3 Skeleton code representing a functional block wrapped by an FMU.

IV. Related Work

A collection of work exists which describes the use of Wasm in Internet of Things (IoT). In [29], Gadepalli et al. present Sledge, a lightweight Wasm runtime designed for edge computing, leveraging Wasm's sandboxing features to enable rapid and isolated execution of stateless functions. The system employs an Ahead-of-Time (AoT) compiler, called aWasm, to produce sandboxed code that performs closely to native execution, enhancing efficiency. Its architecture utilizes asynchronous and per-core scheduling to optimize latency and throughput. Despite these strengths, the approach faces limitations. It is primarily suited for lightweight, event-driven Tasks, which may not extend well to compute-intensive workloads common in avionics. The reliance on POSIX system calls may restrict deployment on specialized hardware used in safety-critical systems. Wen et al. present Wasmachine [30], an operating system designed to efficiently and securely run Wasm applications in resource-constrained environments such as IoT and fog devices. Its primary methodology also involves AoT compilation, translating Wasm binaries into optimized native machine code before execution. This process compiles into native binaries tailored for specific hardware architectures, enabling applications to run as trusted kernel threads. By executing in kernel mode and utilizing Wasm's inherent sandboxing and Rust's memory safety features, Wasmachine reduces the overhead typically associated with system calls and context switching. While this approach offers notable performance gains and enhanced security, it also introduces limitations. The reliance on AoT compilation can prolong development cycles and limit flexibility for dynamic updates. Its focus on IoT and fog use cases restricts its applicability in high-criticality environments, such as avionics. Another related publication from the IoT domain is [31].

An operating system which uses similar concepts as in ISLA is Deos. DDC-I Deos is an ARINC 653-compliant real-time operating system that is designed for safety-critical avionics systems, providing both temporal and spatial isolation through partitioning. These partitions are statically scheduled using a priority-based model, ensuring deterministic real-time performance within each partition. A key feature of Deos is dynamically linked executables. This can be seen as a weak analogy to the interpretation mechanism previously discussed. This increases the reuse of EOC, though not as much as with Wasm. However, because of dynamic linking, at least on the same CPU architecture the very same EOC can be used. Moreover, Deos manages slack time–the unused processing time within a partition. This slack time can be explicitly allocated by processes to other partitions, enhancing overall system resource utilization. However, this model requires explicit requests for slack time, meaning processes must compete for its use, which can introduce complexity. In contrast, in ISLA Tasks are unaware of slack time usage. There is no direct competition for slack, and Tasks automatically benefit from available slack without explicit management. Furthermore, the primary role of slack time in this alternative architecture is not to optimize resource utilization but to reduce Task interference, ensuring that Tasks are not impacted by less time-sensitive ones [32–34].

Another technology related to the concepts from the paper at hand is the Plug and Fly Avionics (PAFA). PAFA, which has been described in several publications [35–37], represents a shift towards autonomous, modular avionics systems, enabling self-configuration and reconfiguration during flight. Unlike traditional systems that rely on static setups, PAFA's self-describing modules dynamically communicate their capabilities and requirements, facilitating flexible, scalable, and fault-tolerant architectures. Central to PAFA is the use of autonomous computing principles which drives the system's dynamic adaptation and fault management. This autonomy allows for runtime reconfiguration, reducing human intervention and enhancing operational flexibility. Additionally, PAFA incorporates a virtual qualification authority which enables continuous qualification of modules and eases certification processes for new components. Like in ISLA, PAFA deploys stateless Tasks without internal state. However, several challenges remain. The continuous exchange of self-descriptive data may introduce performance overhead, which could impact real-time responsiveness, particularly in safety-critical environments. Moreover, achieving fully automated runtime verification and security assessments remains a significant hurdle. While PAFA's conceptual framework offers promising solutions for autonomous avionics, ensuring compliance with existing certification standards such as DO-178C and DO-297, as well as addressing cybersecurity risks, requires ongoing research and development.

V. Conclusion

The ISLA architecture introduced in this paper represents a significant departure from traditional avionics system designs by incorporating serverless computing principles into the safety-critical aviation domain. Its modular, data-driven structure, with stateless and granular Tasks, offers enhanced flexibility and scalability compared to conventional architectures like IMA. ISLA simplifies deployment, maintenance, and system evolution, facilitating adaptation to emerging technological and operational requirements in modern aircraft systems. These advancements are essential for addressing the evolving technological demands and operational scenarios faced in modern avionics.

An enabler of this approach is the use of MBE methodologies, including SysML v2 for high-level modeling, standardized libraries, and automated tools for code generation. These methods streamline system modeling, configuration, and deployment processes, reducing development time and certification efforts. The ability to generate validated and certifiable code directly from high-level models also has the potential to significantly reduce certification costs.

However, several challenges still remain. Compliance with certification standards such as DO-178C, particularly with regard to the verification and validation of a dynamic, stateless system, must be addressed. The implications of dynamic resource allocation and Task self-containment for diagnosis, fault isolation, and security require further comprehensive investigation. Moreover, the qualification strategy for the tools introduced in the MBE context needs to be aligned. The implications of using one qualified tool to verify outputs from the modeling toolchain for further data dissemination still require discussion. In this regard, DO-330, a supplement to DO-178C, DO-254, and DO-278A, should be examined more closely for software tool qualification.

Future research will focus on validating the architecture along with its development toolchain in real-world operational settings, exploring the integration into existing frameworks. Also, adaptive Task scheduling mechanisms while maintaining system predictability and traceability shall be investigated. Further work on safety concerns, particularly around system behavior and data integrity will be essential to ensure the architecture's robustness and suitability for safety-critical applications.

References

[1] Rajan, R. A. P., "Serverless Architecture - A Revolution in Cloud Computing," *2018 Tenth International Conference on Advanced Computing (ICoAC)*, 2018, pp. 88–93. https://doi.org/10.1109/ICoAC44903.2018.8939081.

[2] Li, Z., Guo, L., Cheng, J., Chen, Q., He, B., and Guo, M., "The Serverless Computing Survey: A Technical Primer for Design Architecture," *ACM Comput. Surv.*, Vol. 54, No. 10s, 2022, pp. 1–34. https://doi.org/10.1145/3508360.

[3] Barcelona-Pons, D., Sánchez-Artigas, M., París, G., Sutra, P., and García-López, P., "On the FaaS Track: Building Stateful Distributed Applications with Serverless Architectures," *Proceedings of the 20th International Middleware Conference*, Association for Computing Machinery, New York, NY, USA, 2019, pp. 41–54. https://doi.org/10.1145/3361525.3361535.

[4] RTCA, "Integrated Modular Avionics (IMA) Development Guidance and Certification Considerations," Standard, RTCA, Washington, D.C., USA, Nov. 2005.

[5] Friedrich, S., Schubert, T., Hoche, M. W., Bär, N., Zaeske, W., Bork, P., and Durak, U., "Integrated Serverless Avionics: Next Generation Architecture for Airborne Software," *2025 AIAA DATC/IEEE 44th Digital Avionics Systems Conference (DASC)*, 2025, pp. 1–10. https://doi.org/10.1109/DASC66011.2025.11257378.

[6] ARINC, "AIRCRAFT DATA NETWORK AVIONICS FULL-DUPLEX SWITCHED ETHERNET NETWORK," Standard, Aeronautical Radio Incorporated, Annapolis, MD, USA, Feb. 2006.

[7] Zaeske, W., Friedrich, S., Schubert, T., and Durak, U., "WebAssembly in Avionics: Decoupling Software from Hardware," *2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, 2023, pp. 1–10. https://doi.org/10.1109/DASC58513.2023.10311207.

[8] Zaeske, W., Önem, A. C., Hartung, F., and Durak, U., "On the Design of a WebAssembly Interpreter for Safety Critical Avionics Applications," *2025 AIAA DATC/IEEE 44th Digital Avionics Systems Conference (DASC)*, 2025, pp. 1–10. https://doi.org/10.1109/DASC66011.2025.11257180.

[9] RTCA and EUROCAE, "Software considerations in airborne systems and equipment certification," Standard, Radio Technical Commission for Aeronautics and European Organization for Civil Aviation Equipment, Malakoff, France, Jan. 2012.

[10] Leroy, X., "Formal verification of a realistic compiler," *Commun. ACM*, Vol. 52, No. 7, 2009, pp. 107–115. https://doi.org/10.1145/1538788.1538814.

[11] ARINC, "Avionics Application Software Standard Interface," Standard, Aeronautical Radio Incorporated, Bowie, MD, USA, Dec. 2019.

[12] Kirsch, C. M., and Sokolova, A., *The Logical Execution Time Paradigm*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, Chap. 5, pp. 103–120. https://doi.org/10.1007/978-3-642-24349-3_5.

[13] LAN/MAN Standards Committee, "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks," *IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018)*, 2022, pp. 1–2163. https://doi.org/10.1109/IEEESTD.2022.10004498.

[14] MODELISAR Consortium and The Modelica Association Project FMI, "Functional Mock-up Interface Specification," MAP FMI 5180a79, Oct. 2025. URL https://fmi-standard.org/docs/main/, version 5180a79.

[15] Lukić, B., Friedrich, S., Schubert, T., and Durak, U., *Automated Configuration of ARINC 653-Compliant Avionics Architectures*, American Institute of Aeronautics and Astronautics (AIAA), 2024, Chap. 1856, pp. 1–17. https://doi.org/10.2514/6.2024-1856.

[16] Lukić, B., Friedrich, S., and Durak, U., "A Streamlined Approach Toward Automated Generation and Validation of ARINC 653-Compliant Avionics Configurations," *Journal of Aerospace Information Systems*, Vol. 22, No. 5, 2025, pp. 314–324. https://doi.org/10.2514/1.I011439.

[17] Ahlbrecht, A., Lukić, B., Zaeske, W., and Durak, U., "Exploring SysML v2 for Model-Based Engineering of Safety-Critical Avionics Systems," *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*, 2024, pp. 1–8. https://doi.org/10.1109/DASC62030.2024.10749311.

[18] Bradford, R., Fliginger, S., Nam, M., Mohan, S., Pellizzoni, R., Kim, C., Caccamo, M., and Sha, L., "Exploring the design space of IMA system architectures," *29th Digital Avionics Systems Conference*, 2010, pp. 5.E.51–5.E.515. https://doi.org/10.1109/DASC.2010.5655464.

[19] Januzaj, V., Mauersberger, R., and Biechele, F., "Performance Modelling for Avionics Systems," *Computer Aided Systems Theory - EUROCAST 2009*, edited by R. Moreno-Díaz, F. Pichler, and A. Quesada-Arencibia, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 833–840. https://doi.org/https://doi.org/10.1007/978-3-642-04772-5_107.

[20] Chen, J., Zhai, Z., Han, P., and Huang, M., "Model-based partition scheduling of integrated modular avionics systems using genetic algorithm," *Scientific Reports*, Vol. 15, 2025. https://doi.org/10.1038/s41598-025-16745-4.

[21] Quinton, F., Hamaz, I., and Houssin, L., "A mixed integer linear programming modelling for the flexible cyclic jobshop problem," *Annals of Operations Research*, Vol. 285, No. 1, 2020, pp. 335–352. https://doi.org/10.1007/s10479-019-03387-9.

[22] INCOSE, *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th ed., John Wiley & Sons, Inc., Hoboken, NJ, 2023.

[23] United States Department of Defense, "Defense Acquisition Guidebook," Tech. rep., U.S. Dept. of Defense, Washington, D.C., 2004. URL https://purl.fdlp.gov/GPO/LPS55065.

[24] Snoderly, J., and Faisandier, A., "System Verification," *The Guide to the Systems Engineering Body of Knowledge (SEBoK)*, 2025, pp. 620–628. URL www.sebokwiki.org, version 2.13.

[25] Castelli, C., and Gaudin, E., "XMI, a–not so–standard exchange format," Tech. rep., PragmaDev, Paris, France, 2009. URL https://www.pragmadev.com/news/XMI_fm.pdf.

[26] Ahlbrecht, A., Zaeske, W., and Durak, U., "MBSqlE: Enabling sql as Powerful Query Language for Sysml v2 in Aviation," *2025 AIAA DATC/IEEE 44th Digital Avionics Systems Conference (DASC)*, 2025, pp. 1–10. https://doi.org/10.1109/DASC66011.2025.11257403.

[27] Lukić, B., Beck, J. W., and Durak, U., *Unifying Avionics System Configuration: A Concept for Capturing Essential Elements From Computing Resources to Network Topology in a Universal Format*, American Institute of Aeronautics and Astronautics (AIAA), 2025, Chap. 2515, pp. 1–20. https://doi.org/10.2514/6.2025-2515.

[28] Lukić, B., Nöldeke, P., Durak, U., Klimmek, M., Jakob, S., and Gläser, M., "From Architecture Models to a Target Platform: A Systematic Approach to Model-Driven Development of Dynamically Reconfigurable Avionics Systems," *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*, 2024, pp. 1–10. https://doi.org/10.1109/DASC62030.2024.10749454.

[29] Gadepalli, P. K., McBride, S., Peach, G., Cherkasova, L., and Parmer, G., "Sledge: a Serverless-first, Light-weight Wasm Runtime for the Edge," *Proceedings of the 21st International Middleware Conference*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 265–279. https://doi.org/10.1145/3423211.3425680.

[30] Wen, E., and Weber, G., "Wasmachine: Bring IoT up to Speed with A WebAssembly OS," *2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, 2020, pp. 1–4. https://doi.org/10.1109/PerComWorkshops48775.2020.9156135.

[31] Li, B., Fan, H., Gao, Y., and Dong, W., "ThingSpire OS: a WebAssembly-based IoT operating system for cloud-edge integration," *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, Association for Computing Machinery, New York, NY, USA, 2021, pp. 487–488. https://doi.org/10.1145/3458864.3466910.

[32] DDC-I, "How to Certify Your Code Once, And Use it in Multiple DO-178 Applications," Tech. rep., DDC-I, Nov. 2024. URL https://www.ddci.com/wp-content/uploads/2024/11/How_to_Certify_Once_and_Reuse_with_Deos.pdf.

[33] Intel Corporation, "Delivering Certified, Safety-Critical Computing for the Future of Aviation," Tech. rep., Intel Corporation, Jul. 2023. URL https://www.intel.co.id/content/dam/www/central-libraries/us/en/documents/ddci-deos-rtos-solution-brief.pdf.

[34] Binns, P., "A robust high-performance time partitioning algorithm: the digital engine operating system (DEOS) approach," *20th DASC. 20th Digital Avionics Systems Conference (Cat. No.01CH37219)*, Vol. 1, 2001, pp. 1B6/1–1B6/12 vol.1. https://doi.org/10.1109/DASC.2001.963309.

[35] Annighoefer, B., Riedlinger, M., and Marquardt, O., "How to tell configuration-free integrated modular avionics what to do?!" *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*, 2017, pp. 1–10. https://doi.org/10.1109/DASC.2017.8102064.

[36] Annighoefer, B., Reinhart, J., Brunner, M., and Schulz, B., "The Concept of an Autonomic Avionics Platform and the Resulting Software Engineering Challenges," *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2021, pp. 179–185. https://doi.org/10.1109/SEAMS51251.2021.00031.

[37] Rohrmueller, R., and Annighoefer, B., "Hypervisors and Plug & Fly in a new Space Launcher. A scalable approach to enhance Space Launcher development," *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*, 2024, pp. 1–9. https://doi.org/10.1109/DASC62030.2024.10749521.

[38] ARINC, "SOFTWARE DATA LOADER USING ETHERNET INTERFACE," Standard, Aeronautical Radio Incorporated, Bowie, MD, USA, Jul. 2023.

Appendix

A. Deployment

For the deployment onto the target platform, an atomic image file is loaded onto the target platform. This image contains artifacts from the previous stages of the avionics development process, including compiled software, configuration files, and system-specific settings. Among the various methods of deploying these artifacts, adherence to industry standards ensures both interoperability and reliability. One such standard is ARINC 615A, which specifies the protocols and methodologies for data loading onto airborne systems. This standard enables the loading of software components onto avionics hardware, whether during maintenance on the ground or in-flight.

ARINC 615A

The ARINC 615A standard specifies the uploading and downloading of software and configuration data to and from airborne systems, ensuring the correct operation of avionics hardware in both maintenance and operational contexts. The ARINC 615A standard defines three components for data loading:

Portable Data Loader: A portable system used to load software onto aircraft. It is typically used on the ground or can be carried onboard for in-flight software loading. Modern portable loaders are often based on Commercial Off-The-Shelf ruggedized computing hardware, running specialized software that supports the loading process. Airborne Data Loader: A system permanently installed on the aircraft that facilitates the loading of software during routine maintenance or in-flight updates. This loader operates in a continuous or scheduled manner, ensuring the aircraft's software remains up-to-date and compliant with operational standards. Data Load Function: A software module responsible for executing the data loading process. It can be embedded in the onboard avionics hardware or operate from a portable system. Further, it manages the actual transfer of software and configuration files, ensuring proper system integration and validation.

ARINC 615A specifies several protocols and interfaces necessary for data loading. The communication between avionics systems and the data loader typically occurs over a high-speed Ethernet interface. The data loading protocol relies on multiple protocols: 1. **TFTP** (Trivial File Transfer Protocol) for file transfers during the data loading process. TFTP is used because of its minimal overhead and efficiency in transferring software images to avionics systems. 2. **UDP** (User Datagram Protocol) for data transfer without establishing a reliable end-to-end connection. UDP is preferred for its low latency and suitability for real-time data transfer, though it lacks the guarantees of delivery and error-checking offered by other protocols. 3. **IP** (Internet Protocol) for routing data across networks. IP is used to address and route packets of data between the data loader and the avionics systems. The standard defines various commercial removable media types for software transport, including PC Cards, USB memory devices, and optical discs [38].

*.dts Configuration Format

As a forward-looking concept for future development iterations, the use of Device Tree Source (DTS) files is being considered as a unified configuration mechanism for the avionics platform. While traditionally employed to describe hardware layouts in embedded systems, DTS files may also serve as an expressive and maintainable format for representing the largely static topology of an integrated avionics system.

DTS files could provide an SSOT for relevant system structure, encompassing not only hardware components but also software-defined regions, such as partitioned memory segments, communication channels, and buffers. The hierarchical tree structure of DTS is well-suited for grouping and describing components including applications, partitions, and inter-partition links.

The existing tooling ecosystem–including the device tree compiler, overlay mechanisms, and schema-based validation–offers mature support that could integrate into the build, deployment, and boot processes. The DeviceTree specification's JSON-based binding schemas further enable structural validation, ensuring configuration consistency and reducing the risk of configuration drift throughout the development lifecycle.

Under this concept, the DTS-based configuration could be compiled into an image that is deployed to the target platform alongside other system artifacts. Such an approach would capitalize on DTS's strengths for representing static configurations that rarely change at runtime.

This remains an exploratory concept and is not intended as a near-term design commitment. Its adoption would require further evaluation regarding certification constraints, tooling integration within the avionics development environment, and long-term maintainability. Nonetheless, the approach presents a promising option for future system iterations where centralized and verifiable configuration management is desirable.

```
library package ProfileISLA {
    package ConceptsISLA {
        part def Orchestrator :> Node {
            enum def MemoryType {
                RAM;
                Flash;
                InputOutput;
            }
            enum memoryTypes[1] : MemoryType;

            part def Memory {
                attribute type : MemoryType default = memoryTypes.Flash;
                attribute size default = 1048576;
            }

            part def FailureHandling {
                attribute maxAttempts default = 3;
                attribute backoffStrategy default = "exponential";
                attribute failover default = "backupFunction";
            }

            attribute processing default = "SMP";
            part memories[1..3] : Memory;
            part failureHandlings[1] : FailureHandling;
        }
        part orchestrators[1] : Orchestrator;

        part def DataFabric :> Node {
            attribute version default = 0;

            part def Data {
                attribute mode default = "sample";
                attribute size default = 100;
                attribute depth default = 10;
                attribute interval default = 1000;
            }
            part datas[*] : Data;

            part def Schedule {
                attribute name default = "TheSchedule";
                ...
            }
            ...
        }
        part dataFabrics[1] : DataFabric;

        part def Tasks :> Node {
            ...
        }
    }
}
```

Listing 4 Excerpt of the SysML v2 library for the ISLA platform.

```
package InstanceISLA {
    public import ProfileISLA::ConceptsISLA::*;
    public import ProfileISLA::MetadataISLA::*;

    #node Node {
        :>> architecture : Architecture = architectures.ppc_e500mc_4g;
        :>> memorySize = 1048576;

        #orchestrator ORC {
            :>> processing = "SMP";

            #memory RM {
                :>> type : MemoryType = memoryTypes.Flash;
                :>> size = 1048576;
            }

            #failureHandling FH {
                :>> maxAttempts = 1;
                :>> failover = "backupFunction";
            }
        }

        #dataFabric DataFabric {
            :>> version = 0;

            #data Data {
                :>> mode = "sample";
                :>> size = 100;
            }

            #schedule Schedule {
                :>> name = "TheSchedule";
                :>> core = 1;
                :>> duration = 200;

                #slot Slot {
                    :>> counter = 0;
                    :>> taskName = "Task1";
                    :>> offset = 0;
                }
            }
        }

        #task AntiSkid {
            :>> name = "antiSkid";
            :>> entryPoint = "com.avionics.functions.antiSkid";
            :>> type = "C";
            ...
        }
    }
}
```

Listing 5 Excerpt of an ISLA model instance in SysML v2 textual notation according to the predefined library.

```java
import org.eclipse.xtext.parser.ParseException;
import org.omg.sysml.expressions.util.EvaluationUtil;
import org.omg.sysml.interactive.SysMLInteractive;
import org.omg.sysml.interactive.SysMLInteractiveResult;
...

// Contains the helper functions and profile logic for parsing an ISLA SysML v2 model from the top Node
public class SysMLParsing {
    private Element profile;

    public ArrayList<ArrayList<HashMap<String, Object>>> getPartitions(Element rootElement) {
        rootElement = ((Namespace) rootElement).getOwnedMember().get(0);
        ...
    }

    ...
}

public static void main(String[] args) {
    // Initialize log4j instance
    BasicConfigurator.configure();
    Logger.getRootLogger().setLevel(Level.WARN);

    // Class instance
    SysMLParsing SysMLParser = new SysMLParsing();

    // Get singleton instance.
    SysMLInteractive sysml = SysMLInteractive.getInstance();
    sysml.setVerbose(false);

    // Load the KerML and SysML library models
    System.out.println("Loading libraries...");
    sysml.loadLibrary(args[0]);
    ...

    // Process provided SysML v2 file
    SysMLInteractiveResult result = sysml.process(args[1]);

    // Get the top-level named element in the parsed library model instance.
    Element root = result.getRootElement();

    // Set local profile variable to parsed profile and set root to profile instance
    SysMLParser.profile = ((Namespace)root).getOwnedMember().get(0);
    Element top = ((Namespace)root).getOwnedMember().get(1);

    // Perform further manipulation of model from top element down
    ...
}
```

Listing 6 Excerpt of native SysML v2 parser code.

```
{
    "Node": {
        "architecture": "ppc_e500mc_4g",
        "cores": 1,
        "memorySize": 1048576,

        "Orchestrator": {
            "Processing": "SMP",
            "Memory": [
                {
                    "type": "RAM",
                    "size": 512
                }
            ],
            "FailureHandling": {
                "maxAttempts": 3,
                "backoffStrategy": "exponential",
                "failover": "backupFuction"
            },
            "RealTimeConstraints": {
                "priority": "high",
                "deadline": 250,
                "QoS": "strict"
            }
        },

        "Tasks": [
            {
                "name": "antiSkid",
                "version": "1.0",
                "entryPoint": "com.avionics.functions.antiSkid",
                "executionMode": "event-driven",
                "type": "C",
                "PRED": "none",
                "SUCC": [
                    "antiSkid"
                ],
                "inputs": "none",
                "outputs": [
                    "data1",
                    "data2"
                ],
                ...
            }
        ],

        "DataFabric": {
            ...
        }
    }
}
```

Listing 7 Sample configuration for the ISLA platform in JSON format.